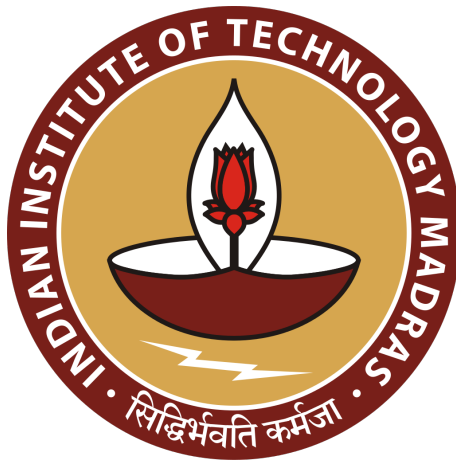


Lab Assignment - 1

Learning Emboss and Seq2Feature



BT3040 - Bioinformatics

Even Semester 2026

Course Instructor: Professor Michael Gromiha

K.Varunkumar | BE23B016

Department of Biotechnology

Indian Institute of Technology, Madras

Question 1

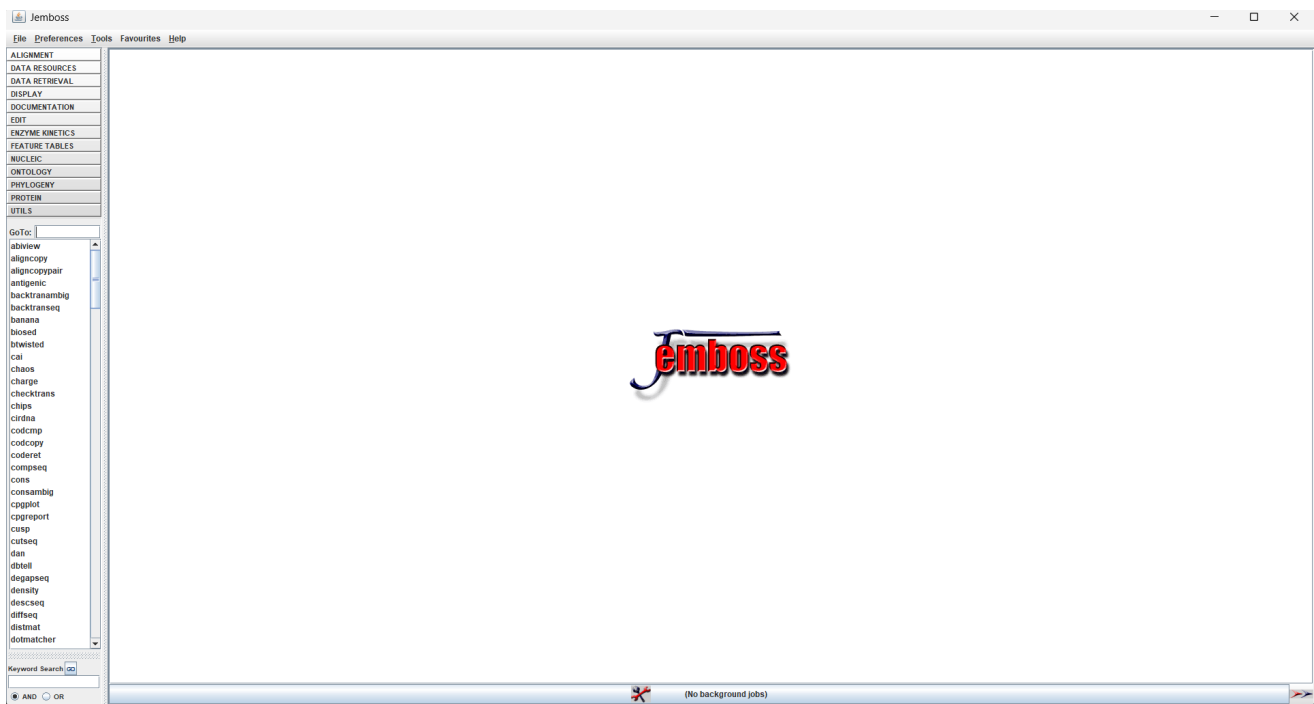


Figure 1.1: EMBOSS on Windows

Question 2

We shall utilize the *REVSEQ* tool in EMBOSS to obtain the complementary strand of the given sequence. The sequence is reversed because DNA is conventionally read in the 5' to 3' direction. Accordingly, if the input sequence is assumed to be oriented from 5' to 3' (left to right), the reversed output is also presented in the 5' to 3' orientation.

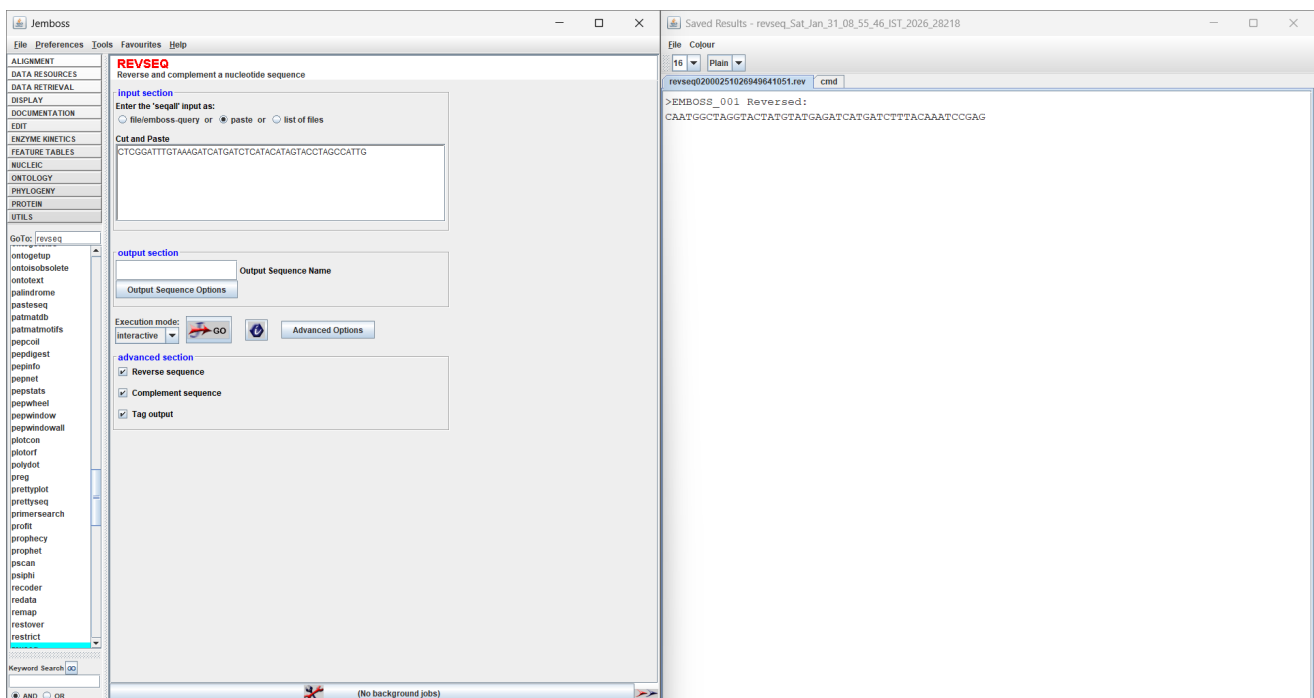


Figure 2.1: Obtaining complementary strand using *REVSEQ*

Question 3

Below is the python code to perform the same function as *REVSEQ* in Question 2

```
# Question 3
def find_complement(seq):

    map = {'A': 'T', 'T': 'A', 'G': 'C', 'C': 'G'}

    seq = seq.upper()
    comp_seq = ""

    for nt in seq:
        comp_seq += map[nt]

    comp_seq = comp_seq[::-1]

    return(comp_seq)

seq = "CTCGGATTTGTAAAGATCATGATCTCATACATAGTACCTAGCCATTG"
complement = find_complement(seq)
print(seq)
print(complement)
```

The output generated is given below

```
CTCGGATTTGTAAAGATCATGATCTCATACATAGTACCTAGCCATTG
CAATGGCTAGGTACTATGTATGAGATCATGATCTTTACAAATCCGAG
```

Question 4

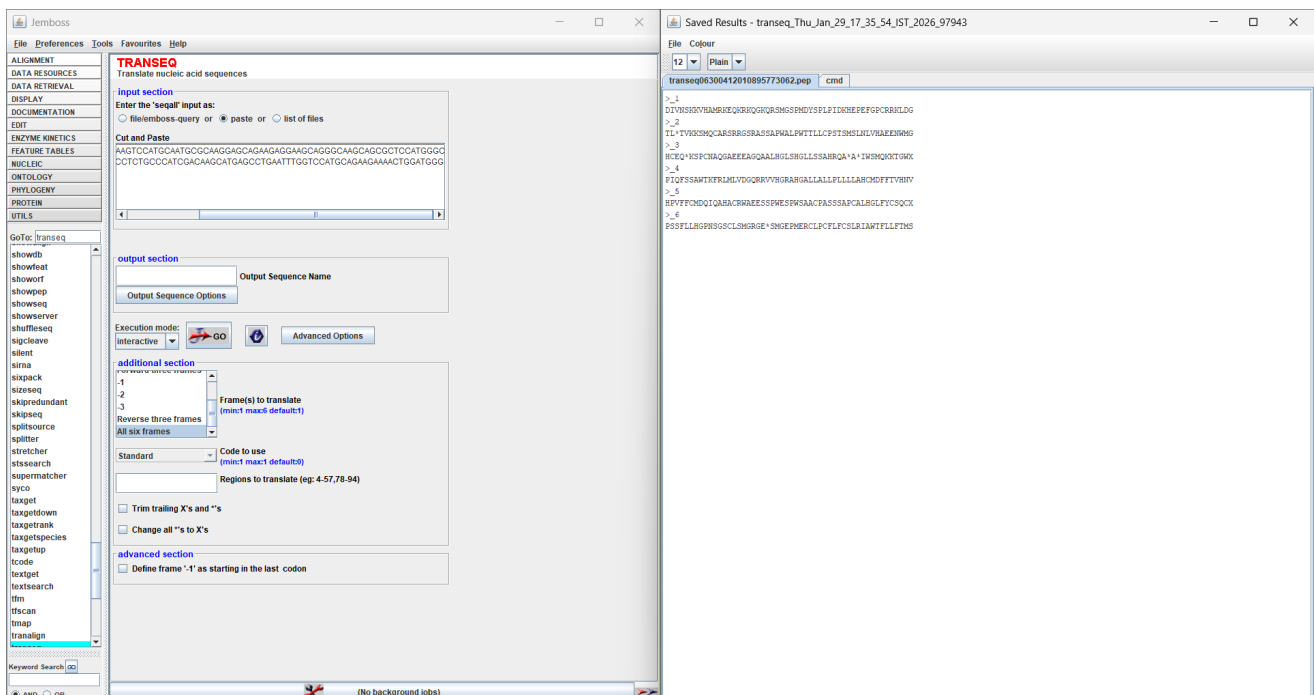


Figure 4.1: Protein sequences from all possible reading frames using *TRANSEQ*

Question 4.1

Figure 4.1 shows the protein sequences obtained from each reading frame.

Question 4.2

The required reading frame in our case is **reading frame 4** which is the *first reading frame in the complementary strand*

Question 5

Below is the python code to perform the same function as *TRANSEQ* in Question 4

```
# Question 5
def translation(seq):

    codon_table = {
        'ATA':'I', 'ATC':'I', 'ATT':'I', 'ATG':'M',
        'ACA':'T', 'ACC':'T', 'ACG':'T', 'ACT':'T',
        'AAC':'N', 'AAT':'N', 'AAA':'K', 'AAG':'K',
        'AGC':'S', 'AGT':'S', 'AGA':'R', 'AGG':'R',
        'CTA':'L', 'CTC':'L', 'CTG':'L', 'CTT':'L',
        'CCA':'P', 'CCC':'P', 'CCG':'P', 'CCT':'P',
        'CAC':'H', 'CAT':'H', 'CAA':'Q', 'CAG':'Q',
        'CGA':'R', 'CGC':'R', 'CGG':'R', 'CGT':'R',
        'GTA':'V', 'GTC':'V', 'GTG':'V', 'GTT':'V',
        'GCA':'A', 'GCC':'A', 'GCG':'A', 'GCT':'A',
        'GAC':'D', 'GAT':'D', 'GAA':'E', 'GAG':'E',
        'GGA':'G', 'GGC':'G', 'GGG':'G', 'GGT':'G',
        'TCA':'S', 'TCC':'S', 'TCG':'S', 'TCT':'S',
        'TTC':'F', 'TTT':'F', 'TTA':'L', 'TTG':'L',
        'TAC':'Y', 'TAT':'Y', 'TAA':'*', 'TAG':'*',
        'TGC':'C', 'TGT':'C', 'TGA':'*', 'TGG':'W',
        'CT' : 'L', 'GT' : 'V', 'TC' : 'S', 'CC' : 'P',
        'AC' : 'T', 'GC' : 'A', 'GC' : 'R', 'GG' : 'G'}

    translations = {}
    seq1,seq2,seq3 = "", "", ""

    for nt in range (0,len(seq),3):

        # Forward Frame 1
        if nt+2 < len(seq):
            seq1 += codon_table[seq[nt:nt+3]]

        elif nt+1 < len(seq):

            if codon_table.get(seq[nt:nt+2]):
                seq1 += codon_table[seq[nt:nt+2]]
            else:
                print(seq[nt+2:nt+4])
                seq1 += "X"

        else:
```

```

        seq1 += "X"

# Forward Frame 2
if nt+3 < len(seq):
    seq2 += codon_table[seq[nt+1:nt+4]]

elif nt+2 < len(seq):

    if codon_table.get(seq[nt+1:nt+3]):
        seq2 += codon_table[seq[nt+1:nt+3]]
    else:
        print(seq[nt+1:nt+3])
        seq2 += "X"

else:
    seq2 += "X"

# Forward Frame 3
if nt+4 < len(seq):
    seq3 += codon_table[seq[nt+2:nt+5]]

elif nt+3 < len(seq):

    if codon_table.get(seq[nt+2:nt+4]):
        seq3 += codon_table[seq[nt+2:nt+4]]
    else:
        print(seq[nt+2:nt+4])
        seq3 += "X"

else:
    seq3 += "X"

translations[1] = seq1
translations[2] = seq2
translations[3] = seq3

seq = find_complement(seq)
seq1,seq2,seq3 = "", "", ""

for nt in range (0,len(seq),3):

    # Reverse Frame 1
    if nt+2 < len(seq):
        seq1 += codon_table[seq[nt:nt+3]]

    elif nt+1 < len(seq):

        if codon_table.get(seq[nt:nt+2]):
            seq1 += codon_table[seq[nt:nt+2]]
        else:
            print(seq[nt+2:nt+4])
            seq1 += "X"

```

```

else:
    seq1 += "X"

# Reverse Frame 2
if nt+3 < len(seq):
    seq2 += codon_table[seq[nt+1:nt+4]]

elif nt+2 < len(seq):

    if codon_table.get(seq[nt+1:nt+3]):
        seq2 += codon_table[seq[nt+1:nt+3]]
    else:
        print(seq[nt+1:nt+3])
        seq2 += "X"

else:
    seq2 += "X"

# Reverse Frame 3
if nt+4 < len(seq):
    seq3 += codon_table[seq[nt+2:nt+5]]

elif nt+3 < len(seq):

    if codon_table.get(seq[nt+2:nt+4]):
        seq3 += codon_table[seq[nt+2:nt+4]]
    else:
        print(seq[nt+2:nt+4])
        seq3 += "X"

else:
    seq3 += "X"

translations[4] = seq1
translations[5] = seq2
translations[6] = seq3

return(translations)

import pprint
seq='GACATTGTGAACAGTAAAAAAGTCCATGCAATGCGCAAGGAGCAGAAGAGGAAGCAGGGCAAGCAGCGCTCCATG
GGCTCTCCCATGGACTACTCTCTCTGCCATCGACAAGCATGAGCCTGAATTTGGTCCATGCAGAAGAAAAGTGGATGGG'
pprint.pprint(translation(seq))

```

The output generated is given below

```

{1: 'DIVNSKKVHAMRKEQKRKQKQKQSMGSPMDYSPLPIDKHEPEFGPCRRKLDG',
 2: 'TL*TVKKSMQCARSRRGSRASSAPWALPWTLLCPSTSMSLNLVHAEENWMG',
 3: 'HCEQ*KSPCNAQGAEEEEAGQAALHGLSHGLLSSAHRQA*A*IWSMQKKTGWX',
 4: 'PIQFSSAWTKFRLMLVDGQRRVVHGRAHGALLALLPLLLLAHCMDFFTVHNV',
 5: 'PSSFLLHGPNSGSLSMGRGE*SMGEPMERCLPCFLFCSLRIAFTLLFTMS',
 6: 'HPVFFCMDQIQAHACRWAEESPWESPWSAACPASSAPCALHGLFYCSQCX'}

```

Question 6

```
# Question 6
# Assuming first occurrence and no overlap
def search_1(seq,subseq):

    locations, start = [], 0

    while seq[start:].find(subseq) != -1:
        ind = (seq[start:].find(subseq))
        locations.append(str(start+ind))
        start += ind + len(subseq)

    print("Total match(es):", len(locations))
    print(f"Position(s) of match:{', ' .join(locations)}")
    return

# Assuming overlap is allowed
def search_2(seq,subseq):

    locations, start = [], 0

    while seq[start:].find(subseq) != -1:
        ind = (seq[start:].find(subseq))
        locations.append(str(start+ind))
        start += ind + 1

    print("Total match(es):", len(locations))
    print(f"Position(s) of match:{', ' .join(locations)}")
    return

seq='GACATTGTGAACAGTAAAAAAGTCCATGCAATGCGCAAGGAGCAGAAGAGGAAGCAGGGCAAGCAGCGCTCCATG
GGCTCTCCCATGGACTACTCTCCTCTGCCATCGACAAGCATGAGCCTGAATTTGGTCCATGCAGAAGAAAAGTGGATGGG'
subseq = input("Enter the sequence to be searched: ").upper()
search_1(seq,subseq)
print("="*125)
search_2(seq,subseq)
```

The output for each test case using both methods is given below:

```
Enter the sequence to be searched: AAG
Total match(es): 7
Position(s) of match:19, 36, 45, 51, 60, 111, 140
=====
Total match(es): 7
Position(s) of match:19, 36, 45, 51, 60, 111, 140
```

```
Enter the sequence to be searched: GTC
Total match(es): 2
Position(s) of match:21, 130
=====
```

```
Total match(es): 2
Position(s) of match:21, 130
```

```
Enter the sequence to be searched: GAG
```

```
Total match(es): 3
Position(s) of match:39, 47, 117
```

```
=====
Total match(es): 3
Position(s) of match:39, 47, 117
```

```
Enter the sequence to be searched: ACTA
```

```
Total match(es): 1
Position(s) of match:88
```

```
=====
Total match(es): 1
Position(s) of match:88
```

```
Enter the sequence to be searched: ATAT
```

```
Total match(es): 0
Position(s) of match:
```

```
=====
Total match(es): 0
Position(s) of match:
```

In the above test cases both methods give the same output, given below is an example which elucidates the difference between the two methods.

```
# Showing the difference between the 2 methods
search_1("ATATAT", "ATAT") # A nucleotide can be a part of only 1 sequence
print("="*125)
search_2("ATATAT", "ATAT") # A nucleotide can be a part of more than 1 sequence
```

This produces the output shown below:

```
Total match(es): 1
Position(s) of match:0
```

```
=====
Total match(es): 2
Position(s) of match:0, 2
```

Question 7

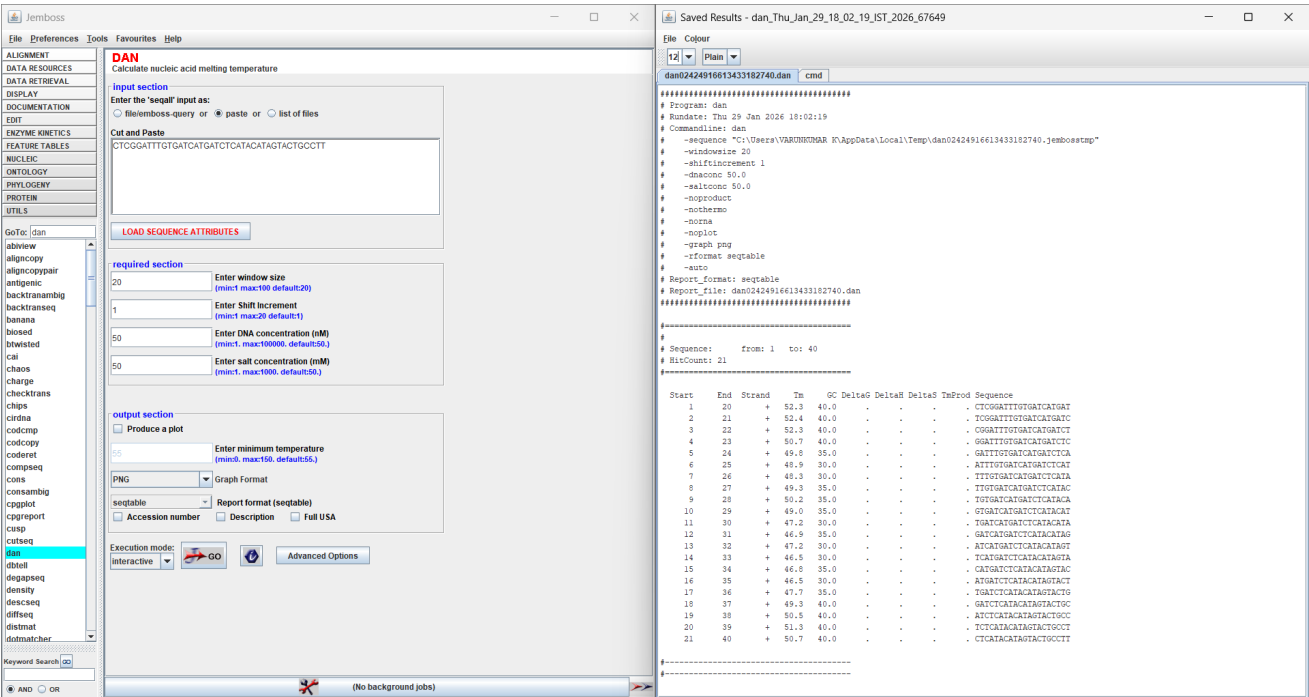


Figure 7.1: Calculating the melting temperature using *DAN*

DAN calculates the melting temperature (T_m) and GC content over a sliding window along the given sequence, thereby enabling the identification of regions suitable for primer design, etc.

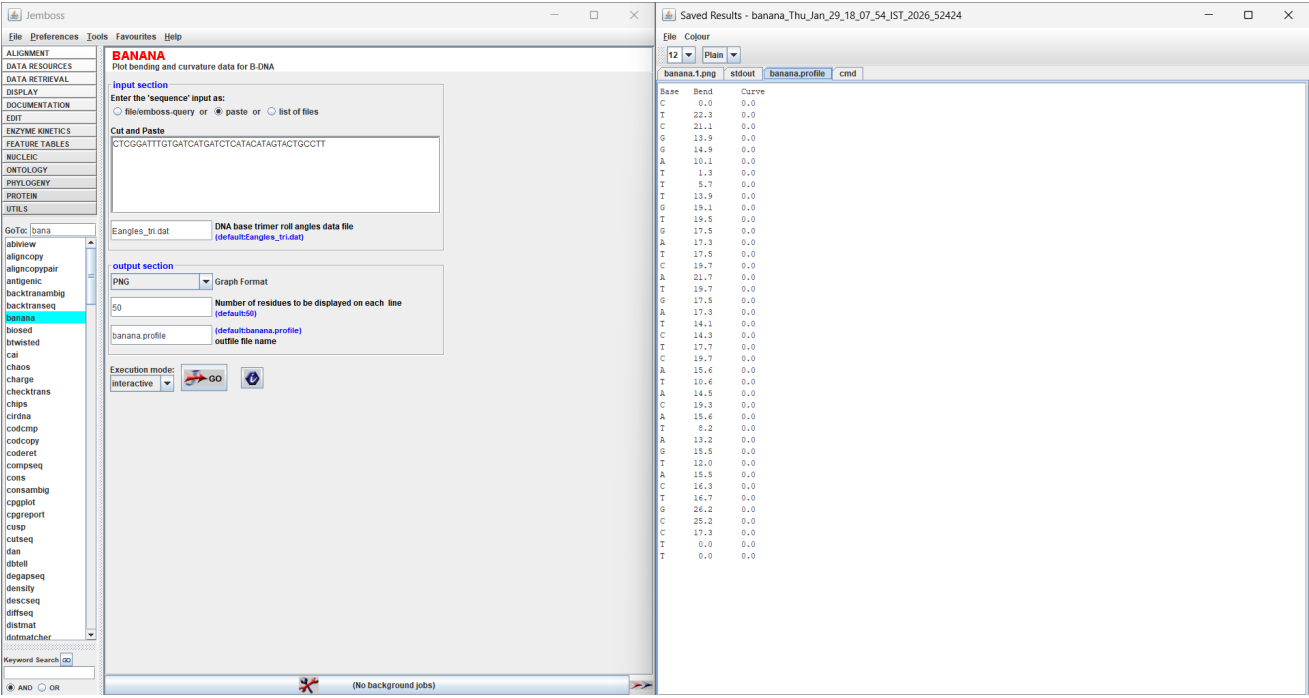


Figure 7.2.a: Determining the bending, curvature using *BANANA*

BANANA analyzes a nucleotide sequence and calculates the magnitude of local bending and macroscopic curvature at each point. The given sequence is too short in our case and hence no curvature is seen, the graphical representation of bending is shown in Figure 7.2.2 below.

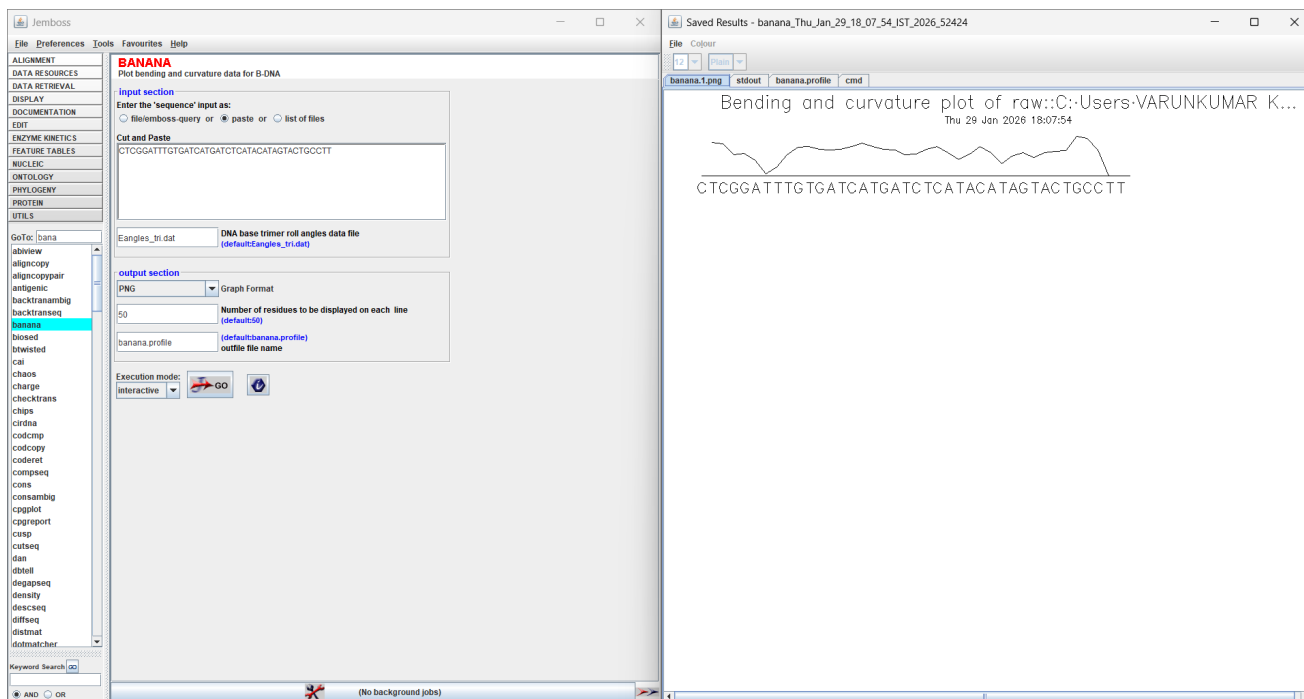


Figure 7.2.b: Bending and curvature plot using *BANANA*

PEPSTATS calculates various physicochemical properties of a protein sequence, including molecular weight, isoelectric point (pI), amino-acid composition etc. These statistics provide insight into the protein's overall biochemical characteristics and structural tendencies.

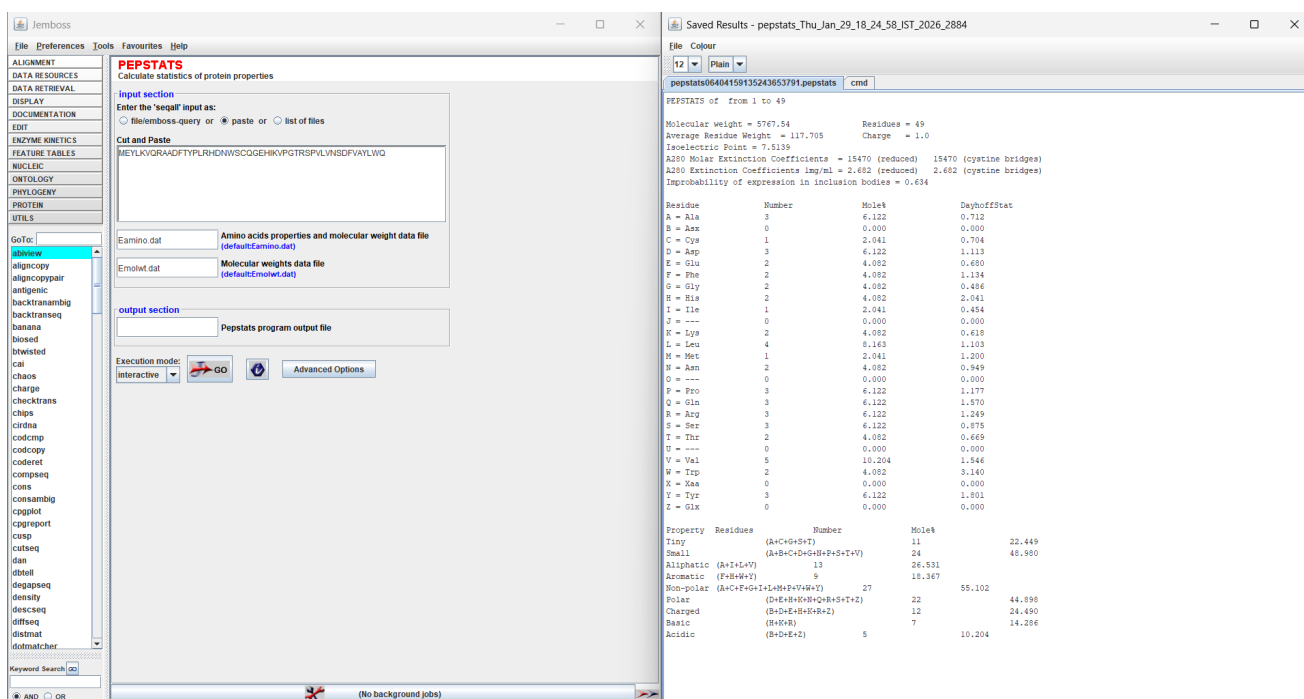


Figure 7.3: Estimating protein sequence's biochemical properties using *PEPSTATS*

EXTRACTSEQ is used to extract specific regions from a nucleotide or protein sequence based on user-defined coordinate ranges.

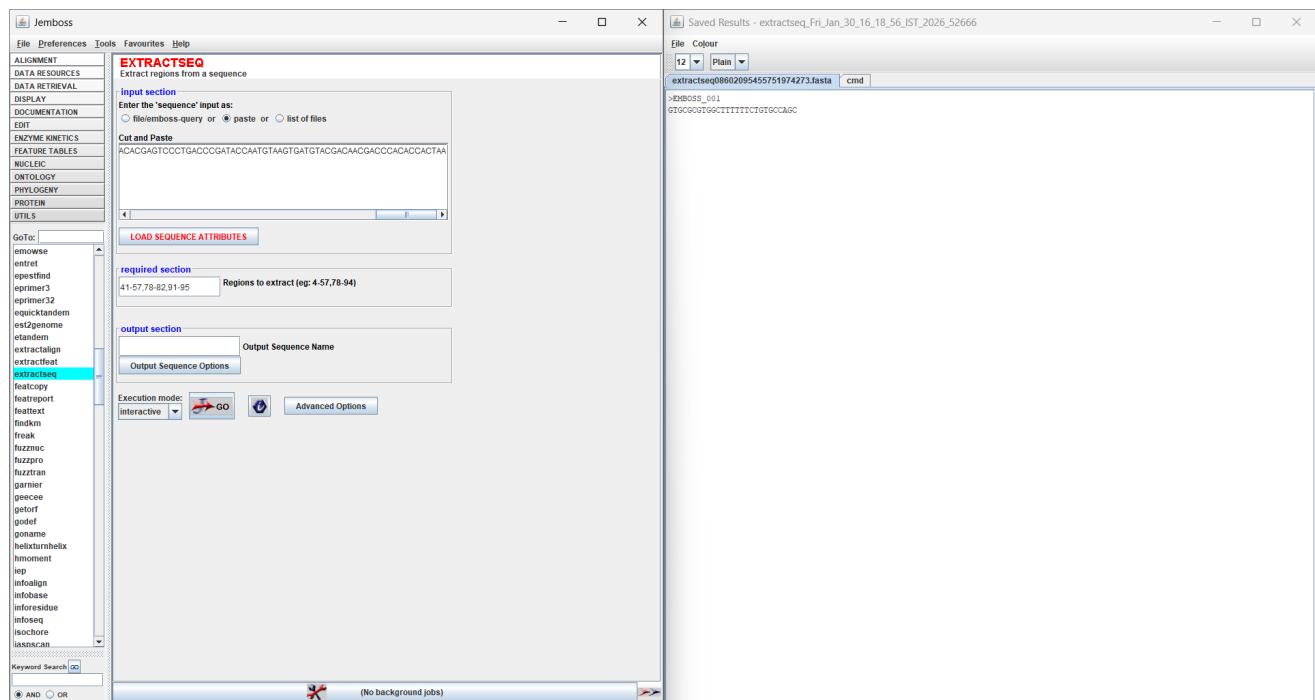


Figure 7.4: Extracting subsequences from a main sequence using *EXTRACTSEQ*

FREAK generates residue or base frequency profiles across a sequence using a sliding window approach. Depending on the analytical requirement, shorter windows can be used to identify motifs such as TATA boxes, while longer windows facilitate the detection of CpG islands and other compositional features.

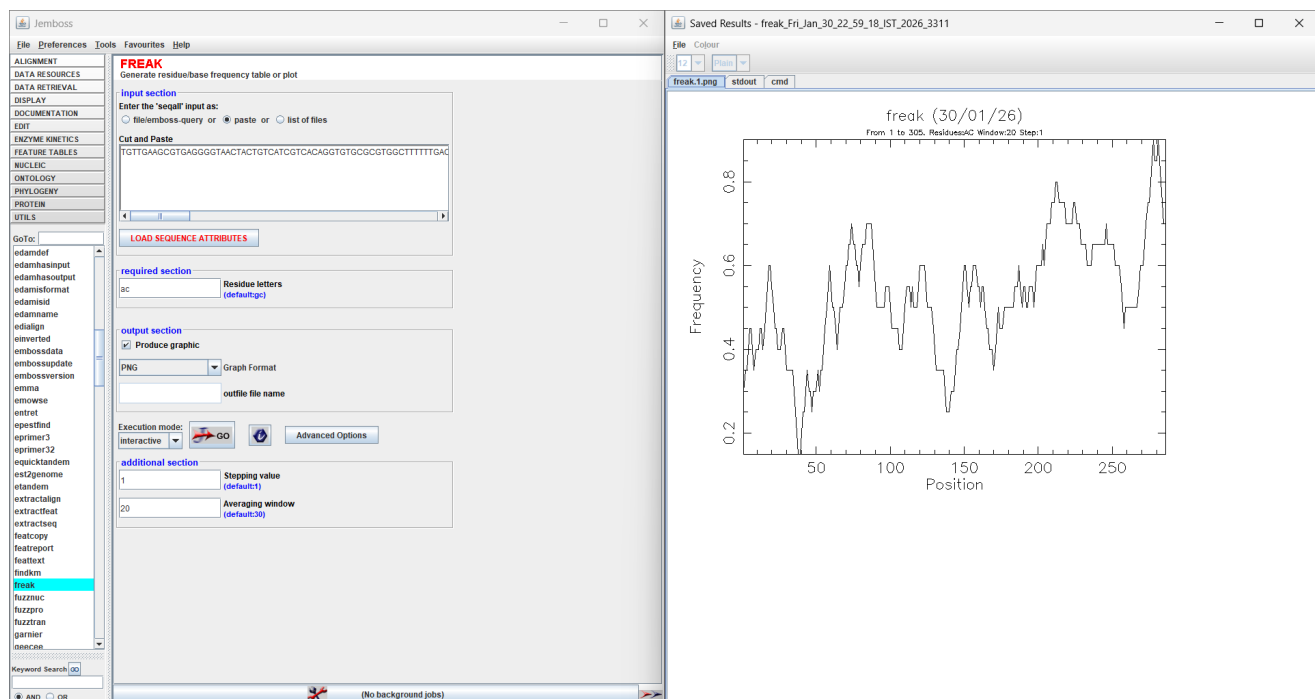


Figure 7.5: obtaining the frequency of adenine and cytosine in a random sequence using *FREAK*

GARNIER predicts the secondary structure of a protein sequence. The output classifies residues into helices, sheets, turns, and coils, providing an initial estimate of the protein’s structural organization.

In the example below, *GARNIER* was utilized on the amino acid sequences of the two chains of the insulin protein to analyze their secondary structures.

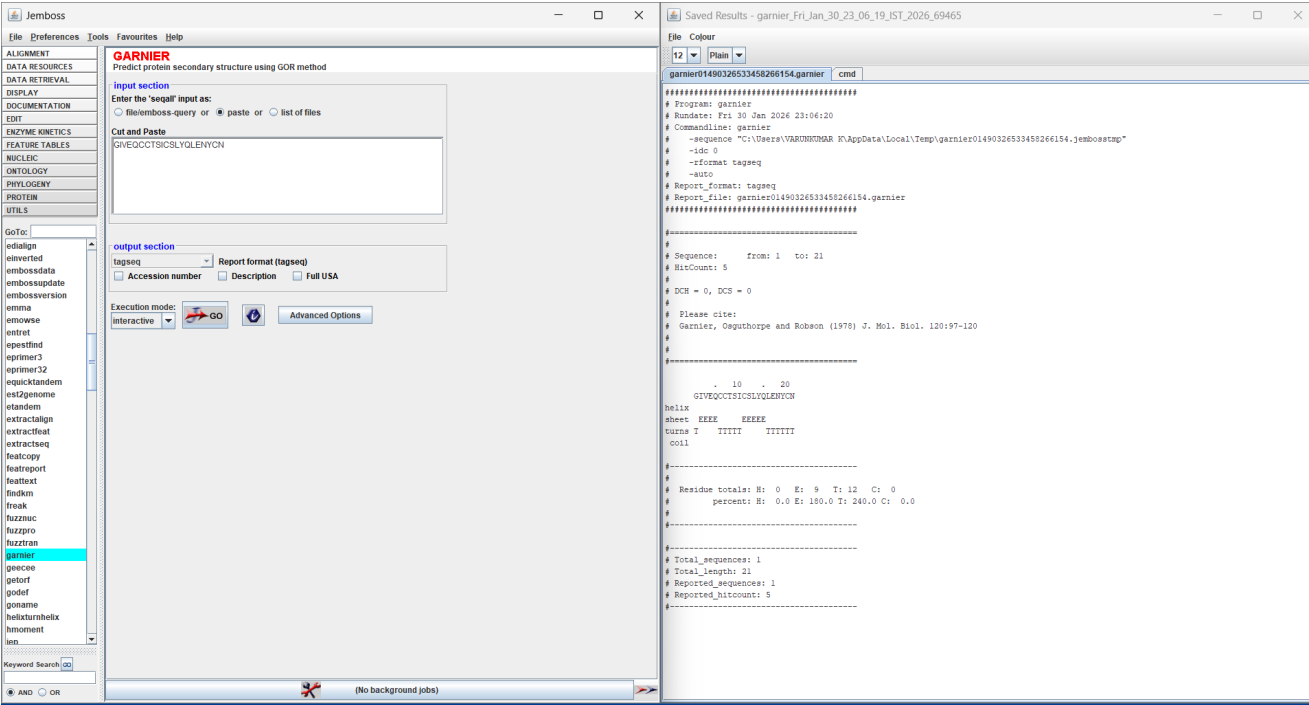


Figure 7.6.1: Insulin A chain, secondary structure analysis using *GARNIER*

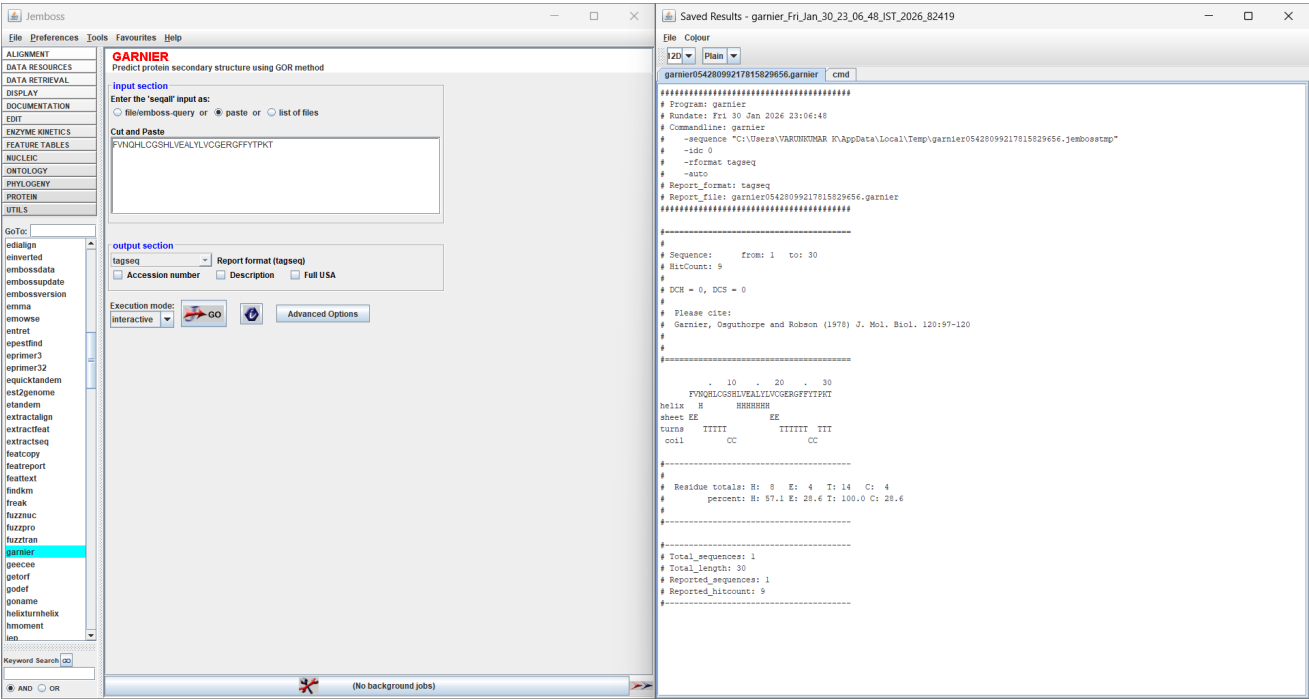


Figure 7.6.1: Insulin B chain, secondary structure analysis using *GARNIER*

Question 8

The python code to compute average base stacking energy is given below.

```
# Question 8 (bse - base stacking energy)
def average_bse(seq):

    bse_table = {
        "AA": -4, "AT": -7, "AC": -5, "AG": -11,
        "TA": -7, "TT": -2, "TC": -3, "TG": -4,
        "CA": -9, "CT": -5, "CC": -6, "CG": -7,
        "GA": -9, "GT": -6, "GC": -4, "GG": -11
    }

    bse = 0

    for n in range(len(seq)-1):
        bse+=bse_table.get(seq[n:n+2])

    abse = bse/(len(seq)-1)

    return(abse)

seq = "CTCGGATTTGTAAAGATCATGATCTCATACATAGTACCTAGCCATTG"
print(average_bse(seq))
```

The output generated is given below

-6.282608695652174

Question 9

Your input seq is:

ATATATATAT

Physicochemical Properties:

Properties	Scale/unit	Average value
Stacking energy	kcal/mol	1.8
Enthalpy	kcal/mol	6.04444
Entropy	cal/mol/K	16.6222
Flexibility_shift	kJ mol ⁻¹ Å ⁻²	2.53
Flexibility_slide	kJ mol ⁻¹ Å ⁻²	9.66333
Free energy	kcal/mol	0.655556
Melting Temperature	degree	48.0022
Mobility to bend towards major groove	mu	1.09778
Mobility to bend towards minor groove	mu	1.03333
Probability contacting nucleosome core	%	6.75556
Rise stiffness	kcal/mol angstrom	7.80778
Roll stiffness	kcal/mol degree	19.3333
Shift stiffness	kcal/mol angstrom	0.892222
Slide stiffness	kcal/mol angstrom	2.66111
Tilt stiffness	kcal/mol degree	28
Twist stiffness	kcal/mol degree	25.8889

Figure 9.1: Average melting temperature, obtained using *Seq2Feature*

From the above image, the melting temperature of the sequence "ATATATATAT" is 48.022, ° C.

Your input seq is:
GCGCGCGCGC

Physicochemical Properties:

Properties	Scale/unit	Average value
Stacking energy	kcal/mol	1.75556
Enthalpy	kcal/mol	11.0778
Entropy	cal/mol/K	27.5556
Flexibility_shift	kJ mol ⁻¹ AA ⁻²	6.49111
Flexibility_slide	kJ mol ⁻¹ AA ⁻²	4.19778
Free energy	kcal/mol	1.85889
Melting Temperature	Degree	107.867
Mobility to bend towards major groove	mu	0.997778
Mobility to bend towards minor groove	mu	1.20556
Probability contacting nucleosome core	%	3.37778
Rise stiffness	kcal/mol angstrom	8.06333
Roll stiffness	kcal/mol degree	21.5556
Shift stiffness	kcal/mol angstrom	1.14667
Slide stiffness	kcal/mol angstrom	2.33889
Tilt stiffness	kcal/mol degree	31.5556
Twist stiffness	kcal/mol degree	20.1111

Figure 9.2: Average melting temperature, obtained using *Seq2Feature*

From the above image, the melting temperature of the sequence "GCGCGCGCGC" is 107.867°C.

The melting temperature of the 1st sequence is significantly lower than that of the 2nd sequence. This difference arises because the 1st sequence is AT-rich, whereas the 2nd sequence is GC-rich, greater energy required to break the three hydrogen bonds between guanine and cytosine, compared to the two hydrogen bonds between adenine and thymine.

Question 10

Your input seq is:
AAATGGCCCTAA

Nucleotide Content:

	Nucleotide content in %
AT_content	58.333333
Adenine_content	41.666667
Cytosine_content	25.000000
GC_content	41.666667
Guanine_content	16.666667
Keto_GT_content	33.333333
Purine_AG_content	58.333333
Thymine_content	16.666667

Figure 10: Calculating AT and GC content of a DNA sequence using *Seq2Feature*

The GC content of the given sequence is 41.67 %. The AT content of the sequence is 58.33 %.

Link to the google drive containing the python code used in this assignment: https://drive.google.com/drive/folders/1y37nRHmFDkWhI5FhRm_50PvT5Zc687wB?usp=sharing